МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2
по курсу «Программирование графических процессоров»

Обработка изображений на GPU. Фильтры. Выделение контуров. Метод Робертса (вариант 5)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы — научиться использовать GPU для обработки изображений: разобраться с бинарным форматом картинок и попрактиковаться с текстурной памятью и двухмерной сетью потоков.

Формат такой: бинарный файл, первые 8 байт — ширина w и высота h (целые, little-endian), дальше пиксели построчно, каждый по 4 байта: r, g, b, a. Альфа-канал в расчётах не участвует, при вычислениях допускается ошибка ±1. Ограничения: $w, h < 2^{16}$ и $w \cdot h \le 10^8$.

Мой вариант — №5, выделение контуров методом Робертса. На вход даются две строки: путь к исходному изображению и путь к конечному. Метод Робертса (перекрёстный оператор Робертса) — простейший градиентный оператор выделения контуров. Для каждого пикселя сначала считается его яркость $L = (299 \cdot r + 587 \cdot g + 114 \cdot b)/1000$, потом две разности яркостей по диагоналям: $gx = L(x, y) - L(x+1, y+1)$ и $gy = L(x+1, y) - L(x, y+1)$. Значение пикселя результата — $G = \sqrt{gx^2 + gy^2}$, при переполнении оно клиппится до 255; результат — полутоновое изображение (r = g = b = G, альфа 0). В пограничном случае изображение «расширяется» за свои границы дублированием граничных пикселей: для любых индексов i и j координаты пикселя определяются как $ip = \max(\min(i, h), 1)$, $jp = \max(\min(j, w), 1)$ — то есть при выходе за край берётся ближайший граничный пиксель. Например, для пикселя последнего столбца в качестве «правого соседа» берётся он сам.

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же проводил все замеры времени и готовил картинки для отчёта.

Графический процессор — NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, максимальная сетка (2 147 483 647, 65 535, 65 535), размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS (ядро 6.8.0-84-generic), драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140), g++ 13.3.0. Бенчмарк и конвертацию бинарных картинок в PNG для отчёта запускал скриптами на этой же машине (чистый Python, без сторонних пакетов).

Метод решения

Сначала читаю из stdin две строки — пути к исходному и конечному файлам. Исходный файл открывается в бинарном режиме, заголовок читаю побайтно и собираю little-endian числа. Дальше валидация: $1 \le w, h < 2^{16}$ и $w \cdot h \le 10^8$. Пиксели читаются в массив на хосте (malloc) и копируются в CUDA-массив на устройстве (cudaMallocArray + cudaMemcpy2DToArray), к которому привязывается текстурный объект (точечная выборка, clamp-адресация); под результат выделяется такой же буфер на устройстве (cudaMalloc).

Ядро работает с двухмерной сетью потоков, конфигурация фиксированная: grid(32×32), block(16×16), всего 262 144 нити — от размера картинки не зависит. Каждая нить ходит по пикселям двумерным grid-stride циклом: шаг по x — gridDim.x·blockDim.x, по y — gridDim.y·blockDim.y. Для своего пикселя (x, y) нить читает из текстуры четыре пикселя: сам пиксель, диагонального соседа (x+1, y+1) и соседей (x+1, y) и (x, y+1); чтение идёт через tex2D, а выход за границы изображения обрабатывается режимом адресации cudaAddressModeClamp — текстура возвращает ближайший граничный пиксель, это и есть «расширение» дублированием граничных пикселей. Для каждого из четырёх пикселей в double считается яркость $L = (299 \cdot r + 587 \cdot g + 114 \cdot b)/1000$, потом диагональные разности $gx = L00 - L11$, $gy = L10 - L01$ и $G = \sqrt{(gx^2 + gy^2)}$ с клипом до 255. Результат пишется как полутоновый пиксель (G, G, G, 0).

Потом результат копируется обратно на хост и пишется в выходной файл: заголовок w, h (little-endian) и все пиксели. Все операции с файлами, malloc и все вызовы CUDA проверяются — при любой ошибке печатается «ERROR: …» в stderr, и программа завершается с кодом 0. При написании ориентировался на официальный CUDA C Programming Guide (docs.nvidia.com).

Описание программы

Программа — один файл lab2.cu.

Ядро robertsKernel(cudaTextureObject_t tex, uchar4 *dst, int w, int h): двумерный grid-stride цикл по пикселям (x, y), для каждого через tex2D читаются из текстуры четыре соседних пикселя uchar4, считаются их яркости в double, диагональные разности и корень из суммы квадратов; в выходной пиксель пишется make_uchar4(v, v, v, 0).

Ещё три маленькие вспомогательные функции: fail — печать сообщения об ошибке в stderr и выход, readPath — чтение пути из stdin со срезанием перевода строки, writeLE32 — запись 4-байтового little-endian числа в файл.

main() читает пути, открывает и читает исходный файл, валидирует размеры, выделяет память на хосте и устройстве, создаёт CUDA-массив и текстурный объект, копирует данные, запускает ядро, копирует результат обратно и пишет выходной файл — и всё это с проверкой ошибок.

Типы: uchar4 — пиксель (r, g, b, a по байту), double — промежуточные яркости и градиенты, long long — размеры и смещения (индекс пикселя $y \cdot w + x$ на предельном наборе доходит до 10^8). Рабочая конфигурация — grid(32×32), block(16×16), фиксированная; остальные конфигурации из таблицы ниже — только для замеров.

Результаты

Время ядра мерял событиями CUDA, в таблице — среднее по 10 запускам (перед замерами — ещё 3 прогревочных запуска). Конфигурации записаны как <<<блоки, нити>>>, в скобках — как сетка и блок раскладываются по двум измерениям. Взял три

входных набора: 512×512 (маленький), 4096×4096 (средний) и 10000×10000 (предельный, тут как раз $w \cdot h = 10^8$). Результат каждой конфигурации сверялся с CPU-эталоном (36 проверок, все совпадения в допуске ±1). Для сравнения — CPU-версия (один поток, компилируется тем же nvcc в том же бинарнике).

Конфигурация (блоки × нити)	**Всего нитей**	**Малый 512×512, мкс**	**Средний 4096×4096, мс**	**Предельный 10000×10000, мс**
<<<1, 32>>> (1×1; 32×1)	32	14707,4	738,77	4446,35
<<<1, 64>>> (1×1; 64×1)	64	6832,0	421,39	2527,94
<<<1, 128>>> (1×1; 128×1)	128	5217,9	333,55	2011,52
<<<1, 256>>> (1×1; 256×1)	256	5141,1	331,40	2028,17
<<<1, 512>>> (1×1; 512×1)	512	5191,1	331,52	2026,11
<<<1, 1024>>> (1×1; 32×32)	1 024	5190,8	334,21	2011,93
<<<4, 256>>> (2×2; 16×16)	1 024	1301,6	83,66	503,63
<<<16, 256>>> (4×4; 16×16)	4 096	652,9	41,81	251,91
<<<64, 256>>> (8×8; 16×16)	16 384	409,9	26,14	157,04
<<<256, 256>>> (16×16; 16×16)	65 536	389,4	24,81	148,99
<<<1024, 256>>> (32×32; 16×16)	262 144	379,3	24,16	144,88
<<<1024, 1024>>> (32×32; 32×32)	1 048 576	404,3	24,20	145,15
CPU (1 поток)	1 поток	4787,2	309,73	1842,11

Картина такая. Пока блоков мало, время обратно пропорционально их числу, а после ~16 384 нитей выходит на плато (на маленьком наборе 379–404 мкс). Лучшая конфигурация — <<<1024, 256>>> (32×32; 16×16): 379,3 мкс на маленьком наборе, 24,16 мс на среднем и 144,88 мс на предельном; она же используется в программе. Конфигурация с 1 048 576 нитей на маленьком наборе уже чуть хуже (404,3 мкс против 379,3 мкс) — расходы на

запуск слишком большой сетки. Сеть 1×1 — это один блок, то есть один мультипроцессор из четырнадцати: одномерные конфигурации <<<1, 128>>>…<<<1, 1024>>> дают практически одно и то же время (5,14–5,22 мс на маленьком наборе), его определяет латентность памяти одного мультипроцессора, а не пропускная способность. С 32 нитями GPU в три раза медленнее CPU (14,71 мс против 4,79 мс), но уже при 1 024 нитях в конфигурации <<<4, 256>>> он догоняет CPU, а дальше обгоняет.

Ускорение против одноядерного CPU примерно одинаковое на всех трёх наборах — около 12,7 раз (12,6× на маленьком, 12,8× на среднем и 12,7× на предельном: 144,88 мс против 1842,11 мс). На предельном наборе эффективный трафик — 4 байта чтения + 4 байта записи на пиксель (четырёхкратное перекрытие чтений соседних пикселей поглощается текстурным кэшем), то есть 800 МБ за 144,88 мс — всего около 5,5 ГБ/с при пиковой пропускной способности GTX 1650 порядка 128 ГБ/с. Значит, ядро упирается не в память, а в вычисления: каждая нить считает яркость четырёх пикселей в double, а двойная точность на GeForce выполняется с многократно меньшей скоростью, чем одинарная, и соседние нити при этом пересчитывают одни и те же пиксели заново.

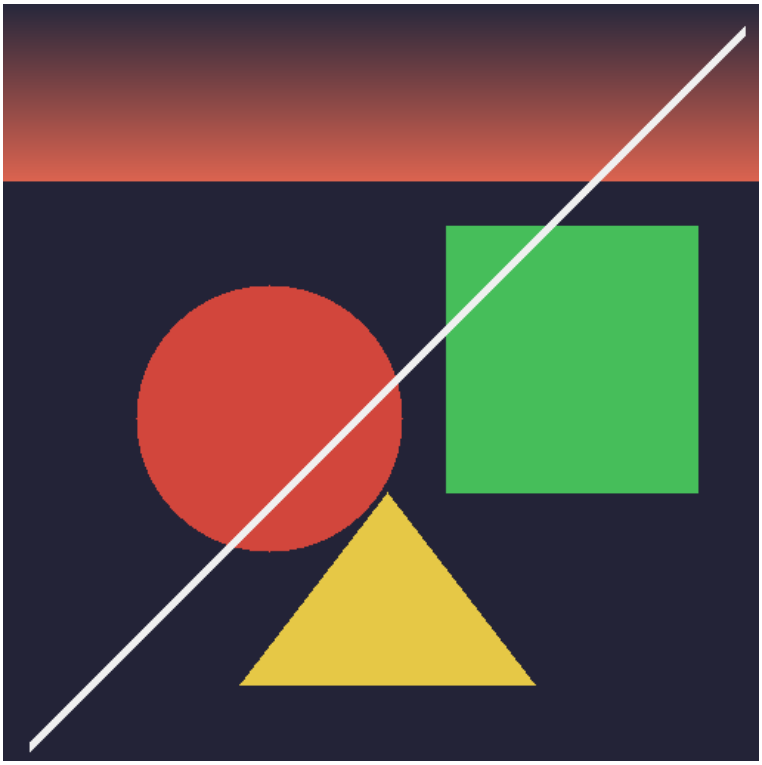


Рисунок 1 — Исходное изображение (512×512)

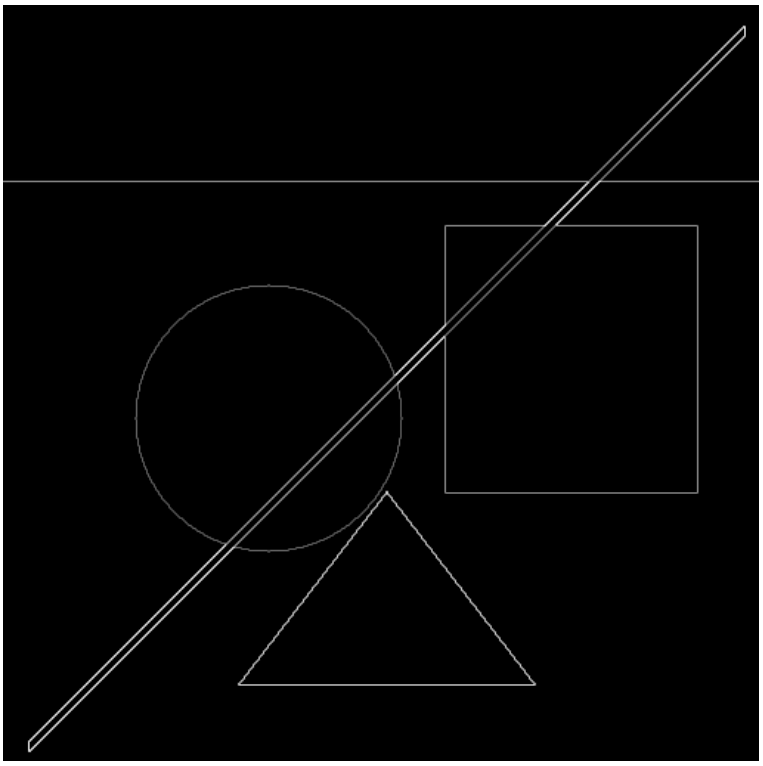


Рисунок 2 — Результат: выделение контуров методом Робертса (512×512)

Выводы

Оператор Робертса — один из простейших градиентных операторов выделения контуров: он аппроксимирует градиент яркости изображения разностями по двум диагоналям. Применяется в обработке изображений и компьютерном зрении для детектирования границ объектов, например как предобработка перед сегментацией или распознаванием; из-за маленького ядра 2×2 оператор чувствителен к шуму, поэтому на зашумлённых изображениях его обычно комбинируют со сглаживанием.

Полученные результаты объясняются так. Производительность растёт с числом нитей лишь до насыщения GPU — примерно 16 384 нити, когда блоков хватает на все 14 мультипроцессоров, — после чего выходит на плато. Ускорение против одноядерной CPU-версии — около 12,7 раз на всех трёх наборах. В отличие от типичных фильтров, ядро не выходит на пик пропускной способности памяти (5,5 ГБ/с из ~128 ГБ/с): оно ограничено вычислением яркости в двойной точности, которое каждая нить выполняет заново для четырёх пикселей. Чтение исходных пикселей идёт через текстурную память (CUDA-массив и текстурный объект с clamp-адресацией): текстурный кэш хорошо ложится на двумерную локальность обращений и сам обрабатывает выход за границы, что дало небольшой выигрыш — около 5%. Возможные оптимизации — целочисленный или одинарный расчёт яркости и переиспользование яркостей соседних пикселей через разделяемую память. Конфигурация запуска важна: при одном блоке работает один

мультипроцессор из четырнадцати, поэтому при малом числе блоков GPU недозагружен и даже проигрывает CPU.